

Day 2: Modules and Binding

SECTIONS :-

- 1) Modules
- 2) Connection between 2 Modules
- 3) One way binding
- 4) Two way binding- Forms Module
- 5) Task

1) Module-

Angular modules are TypeScript classes that group related components together. The `@NgModule` decorator marks a class as an Angular module. The developer decides which components belong inside the module. Angular uses modules to understand how different components are connected. The folder is only for organizing files, not the actual module.

To create a module- `ng g m module-name`

This creates a folder with the name- 'module-name' and inside that a [module.ts](#) file. This [module.ts](#) file has the class which is the actual module. Developers usually keep components belonging to a module in the same folder as the module. But in the end, it's not mandatory. A module is not the folder. It's the class with decorator `@NgModule`.

2)Connection between 2 Modules-

What do you mean by connecting two modules? If you can use components from one module inside components of another module, then the modules are connected.

Suppose you have two modules: mod1 and mod2. Inside mod1 there is a component called comp1, and inside mod2 there is a component called comp2.

If you want to use comp2 inside comp1 (for example, using a Navbar component inside a Home component), you first need to export comp2 from mod2.

To do this, go to the module.ts file of mod2. (See again 🙋 — the module is the module.ts file, not the folder. So when I say "go to the module and export the component", I mean go to that module.ts file.) Add comp2 to the exports list.

Next, go to mod1 (the module.ts file of mod1).

There are two things to do:

1. Import mod2 into mod1.

Example:

```
import { FormsModule } from '@angular/forms';
```

2. Add it to the imports list.

Example:

```
imports: [  
  BrowserModule,  
  AppRoutingModule,  
  FormsModule, // add it here  
  AuthModule,  
  RouterModule,  
  CoreModule  
]
```

After this, you can use comp2 inside any component that belongs to mod1.

To use it in a component, simply place its selector tag inside that component's HTML file.

Summary for this section-

- comp2 belongs to mod2.
- Add comp2 to the exports list of mod2.
- Import mod2 into mod1.
- Add mod2 to the imports list of mod1.
- Use the selector of comp2 inside components of mod1

3) One way binding-

First some basics before starting binding-

(A component is made of three things: the HTML file, the CSS file, and the TypeScript (.ts) file. (For now, you can ignore the spec.ts file.)

These three files are automatically connected by Angular. You do not have to manually link the HTML and CSS files yourself.

If you open your component's .ts file, you will see something like this:

```
@Component({  
  selector: 'app-home',  
  templateUrl: './home.component.html',  
  styleUrls: ['./home.component.css']  
})
```

The `templateUrl` tells Angular which HTML file and `styleUrls` tells Angular which CSS file, belongs to the component.

When you run:

```
ng g c component-name
```

Angular automatically creates these files for you.

Angular also places them inside a folder and automatically connects the HTML and CSS files to the TypeScript file. (Life is so easy because of angular right...just imagine if you have 100's of components. Manually linking them would be hell.)

)

One-Way Binding-

One-Way binding is the connection between your component's .ts file and its HTML file. It allows you to display data from the .ts file in the HTML.

There are three types of One-Way binding:

1. Interpolation ({{ }})

Interpolation is used to display the value of a variable or property (class ke variables ko property bolte hain) in the HTML.

Example:

In the .ts file:

```
name = "Gayatri";
```

In the HTML file:

```
{{ name }}
```

The browser will display:

Gayatri

Notice that the variable is created in the .ts file, not in the HTML file. The HTML file only displays its value.

2. Property Binding ([])

Property binding is used to set the value of an HTML element's property using a variable from the .ts file.

Example:

In the .ts file:

```
imageUrl = "cat.jpg";
```

In the HTML file:

```
<img [src]="imageUrl">
```

Angular takes the value stored in imageUrl and assigns it to the src property of the image.

3. Event Binding (())

Event binding is used when you want something to happen after a user action such as a button click.

Example:

In the HTML file:

```
<button (click)="showMessage()">Click Me
```

In the .ts file:

```
showMessage() {  
  console.log("Button clicked");  
}
```

When the button is clicked, Angular calls the showMessage() function from the .ts file.

Section summary-

❖ One Way binding-

- Interpolation - (`{{ }}`)
- Property Binding - (`[ ]`)
- Event Binding- (`()`)

4) Two way binding- Forms Module:-

Till now, all the bindings we have seen were examples of one-way binding. In one-way binding, data moves only in one direction. For example, interpolation and property binding send data from the .ts file to the HTML file, while event binding sends data from the HTML file to the .ts file.

But sometimes you want both sides to stay updated automatically. For example, suppose you have a textbox and a variable called `name`. If the user types something into the textbox, you want the variable to update automatically. Similarly, if the value of the variable changes in the .ts file, you want the textbox to show the new value automatically.

This is called two-way binding. In two-way binding, data can move from the .ts file to the HTML file and also from the HTML file back to the .ts file.

Angular provides two-way binding using `ngModel`.

Before using `ngModel`, you need to import `FormsModule` and add it to the imports list of your module.

Example:

In the .ts file:

```
name = "";
```

In the HTML file:

```
<input [(ngModel)]="name">
```

Now Angular keeps both the textbox and the variable synchronized. If the user types in the textbox, the variable updates automatically. If the variable changes in the .ts file, the textbox updates automatically.

5) **Task-**

Create a component that contains a login form with two fields: Username and Password.

Use two-way binding (**ngModel**) to bind the input fields to variables in the component's TypeScript file.

When the user clicks the Submit button, check the entered values.

- If the username is "**Bharat**" and the password is "**India**", display an alert saying "**Login Successful**".
- Otherwise, display an alert saying "**Login Unsuccessful**".

Extra:-

These two things were mentioned in Day-2: Constructor vs ngOnInit. You can study it on your own.

THANK YOU.